

EXAMEN FINAL

Ingenieria de Software II

Enrique Lujan

github.com/tunek12421/project-exam-ingenieria-software

Rama: examen-enrique-lujan

A. Parte Teorica

1. Monolitica vs por capas

La monolitica tiene todo junto en un solo bloque. La de capas separa en niveles (presentacion, negocio, datos), cada uno con su responsabilidad. Es mas facil de mantener y probar.

2. Refactorizacion y funcionalidad

La refactorizacion mejora la estructura del codigo sin cambiar lo que hace. Si cambia el comportamiento ya no es refactorizacion. Las pruebas deben seguir pasando igual.

3. Pruebas automatizadas

Detectan errores rapido y sin probar a mano. Cuando cambias algo, te avisan si rompiste algo. Sirven como documentacion del comportamiento esperado.

B. Parte Practica

Actividad 1 - Organizacion y versiones

Python/Flask, HTML/CSS/JS, SQLite, pytest, Docker. Rama examen-enrique-lujan con 11 commits.

Estructura:

backend/src/ -> config, controllers, services, models, repositories
frontend/src/ -> components, pages, services, assets
database/ -> schema.sql (task_lists + subtasks), seed.sql
tests/ -> unitarias (15) e integracion (6)
Dockerfile, docker-compose.yml, README.md

Commits principales:

1. feat: estructura base con modelo, repositorio y BD
2. feat: API REST y frontend
3. test: pruebas unitarias e integracion
4. refactor: correccion de code smells
5. feat: dockerizacion
6. feat: rediseño con listas, subtareas y progreso circular
7. fix: manejo de requests sin JSON valido
8. ui: timeline con linea vertical y dots
9. fix: preservar paneles expandidos al toggle
10. ui: cerrar formulario de subtask al agregar
11. docs: actualizacion README y documento de respuestas

Actividad 2 - Arquitectura

Arquitectura por capas: Frontend -> Controllers -> Services -> Repositories -> SQLite.
Modelos: TaskList (con progreso calculado) y Subtask.

Mejoras posibles: migraciones Alembic, autenticacion JWT, middleware de errores, migrar frontend a Vue/React.

Actividad 3 - Refactorizacion

Problemas encontrados y corregidos:

- Conexiones BD sin cierre seguro -> context manager (with)
- Query SQL repetida 3 veces -> constante `_BASE_SELECT`
- Import datetime sin usar -> eliminado
- Numero 200 hardcoded -> constante `TITLE_MAX_LENGTH`
- Validacion duplicada en service -> metodo `_find_list_or_raise()`
- Sin validacion en frontend -> `trim()` + check vacio
- Sin aviso de exito -> notificaciones toast
- Paneles se cerraban al toggle -> estado preservado con Set
- Request sin JSON daba error 415 -> `get_json(silent=True)` retorna 400

Actividad 4 - Funcionalidad

El sistema permite:

- Crear tareas con titulo y descripcion
- Agregar subtareas dentro de cada tarea (checklist)
- Marcar y desmarcar subtareas (toggle)
- Circulo de progreso segun subtareas completadas
- Expandir/colapsar subtareas con timeline (linea + dots)
- Eliminar tareas (cascade) y subtareas
- Validacion de campos vacios (front y back)
- Paneles se mantienen abiertos al interactuar

Endpoints:

- GET `/api/task-lists` - Listar tareas con subtareas y progreso
- POST `/api/task-lists` - Crear tarea
- DELETE `/api/task-lists/<id>` - Eliminar tarea (cascade)
- POST `/api/task-lists/<id>/subtasks` - Agregar subtask
- PATCH `/api/subtasks/<id>/toggle` - Marcar/desmarcar subtask
- DELETE `/api/subtasks/<id>` - Eliminar subtask

Actividad 5 - Pruebas

21 pruebas (15 unitarias + 6 integracion). Todas pasan.

Unitarias: crear lista, titulo vacio/espacios/largo, eliminar lista, agregar subtask, toggle on/off, subtask inexistente, obtener listas, progreso 0%/25%/100%.

Integracion: crear y listar, error 400 sin titulo, progreso con subtareas, toggle check/uncheck, cascade delete, error 404.

```
$ python3 -m pytest -v
21 passed in 0.39s
```